Native Animation Flow Matching: Generating Stylized Animation from a Single Keyframe

Aaditya Baranwal, Grace Lim, Priyank Pathak, Xin Liang, Dawn Martin

Abstract

We present Native Animation Flow Matching (Native FM), a framework for generating short stylized-animation continuations conditioned on a single keyframe. Off-the-shelf video diffusion and Flow Matching models inherit a strong photorealism prior from their training data, and they reliably collapse the smears, impact frames, and morphing motions that define native animation. We refer to this failure as the realism trap, and we address it on two fronts. First, we curate a Sakugabooru corpus of 11,869 clips covering more than 240 anime series and expand it into approximately 3.6M keyframe-to-continuation pairs through a reproducible windowing pipeline. Second, we extend a Wan-based Flow Matching backbone with three coordinated modifications: a keyframe-preserving scheduler shift, a motion-aware per-frame loss reweighting, and a latent temporal-difference consistency regularizer. On a four-axis evaluation that combines continuation fidelity (CFS), temporal consistency (TCS), worst-segment quality (WorstSeg), and a diffusion-failure score (DFS), the 5B configuration of Native FM raises the FinalScore from 0.50 to 0.65. The improvement is concentrated in collapse-related axes: a 0.20 reduction in DFS and a 0.09 gain in WorstSeg, rather than raw fidelity. The relative training-loss reduction of 14.9% over a single epoch is consistent with what FLUX- and Stable-Diffusion-style proxies achieve on their simpler velocity-MSE objective, despite the strictly harder composite loss optimized by Native FM.

1 Introduction

Modern video generation systems [8, 3, 14] produce visually striking photorealism, yet they consistently fail on native animation, the deliberately exaggerated and physics-violating motion that defines high-quality 2D animation. Trained on web-scale real-world video, these models inherit a strong realism prior in which gravity, inertia, and uniform lighting are baked in. The smears, impact frames, and squash-and-stretch deformations that drive expressive animation are smoothed away as a result. We refer to this failure mode as the realism trap, and Fig. 1 illustrates the kind of stylized motion that is at stake.

This paper studies keyframe-conditioned native-animation generation. Given a single anime frame, the goal is to generate a 1 to 4 second continuation that preserves both the artistic style and the temporal coherence of the source. Our contributions are fourfold:

1. A curated sakuga dataset of 11,869 clips drawn from more than 240 series and annotated with 25 technique categories such as smears, impact frames, and morphing. A reproducible windowing pipeline expands these clips into roughly 3.6M keyframe-to-continuation training pairs.
2. A motion-aware Flow Matching loss that reweights the per-frame velocity error by latent motion magnitude, concentrating model capacity on high-action segments rather than the long static stretches that dominate sakuga.
3. A latent temporal-difference consistency regularizer that supervises frame-to-frame deltas of the predicted clean signal, directly attacking the flicker and mid-clip collapse modes that velocity-only losses ignore.
4. A multi-axis collapse-sensitive evaluation framework comprising continuation fidelity, temporal consistency, worst-segment quality, and a diffusion-failure score. The aggregate FinalScore is calibrated so that a single localized breakdown disproportionately penalizes the clip, mirroring the perceptual fact that one bad frame substantially degrades a viewer's verdict on the full sequence.

On a Wan2.2-TI2V-5B backbone, Native FM lifts the FinalScore from 0.50 on the untuned baseline to 0.65 after one epoch on $\sim$1.2M training pairs, with the gain dominated by a 0.20 reduction in DFS rather than a fidelity rewrite. Cross-family training on FLUX- and SD-style backbones confirms that the relative loss drop on the strictly harder composite objective is consistent with what those proxies achieve on their simpler velocity MSE.

2 Related Work

The most directly relevant prior work falls into three threads. The first is image and video diffusion. Image diffusion [7, 13] extends naturally to video through 3D denoising backbones [8, 3] and, at scale, through transformer-based latent diffusion as in the Wan family [14]. Flow Matching [10] replaces the stochastic denoising trajectory with a deterministic ordinary differ-

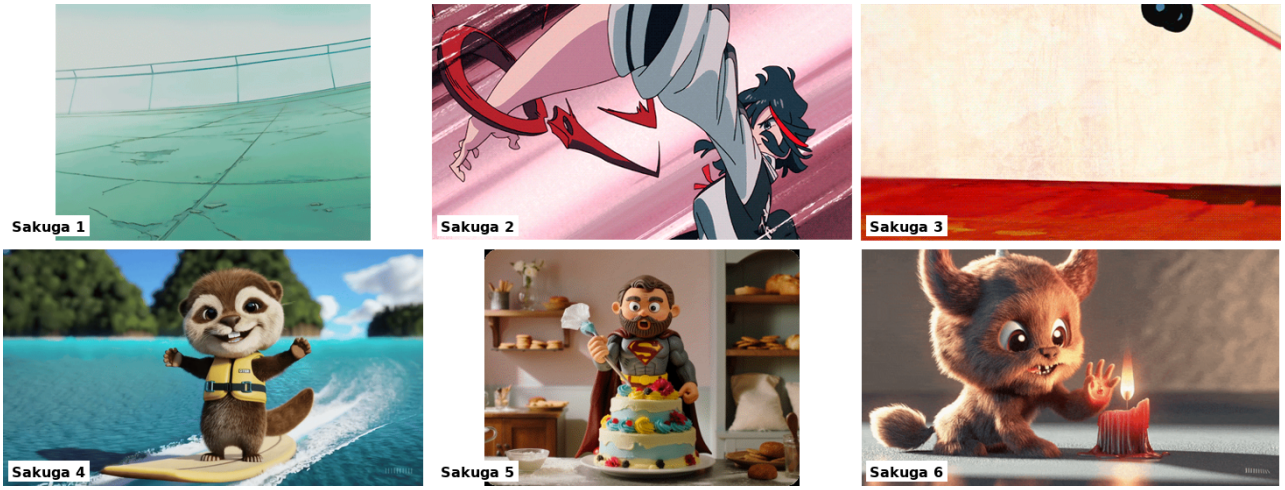

Figure 1: The realism trap. Six representative native-animation clips spanning traditional 2D anime, modern character animation, and 3D shorts. Each contains squash-and-stretch, impact, or morphing motion that violates physical priors and is reliably erased by off-the-shelf photorealistic video models.

ential equation that is faster to sample and easier to train, with closely related formulations including Rectified Flow [11] and stochastic interpolants [1]. The recent SD3 family [4] scales the Flow Matching transformer recipe to high-resolution image synthesis, and the Wan2.x family [14] extends it to video. We use Wan2.2 as our backbone. None of these systems explicitly accounts for the distributional properties of stylized animation, which is the gap Native FM targets.

The second thread is keyframe-conditioned video synthesis. Conditioning a video model on a single keyframe is a well-established interface, with the Wan, Stable Video Diffusion [3], and AnimateDiff [6] families all supporting it. Our setting differs in two respects. We do not modify the conditioning interface; the contribution is purely on the loss and schedule side. We also explicitly target stylized motion rather than photorealistic continuation, which changes which failure modes are most important to address.

The third thread is stylized animation generation, which is comparatively sparse in the literature. AnimateDiff [6] adapts image diffusion models with personalized motion modules but typically operates from text, and it does not address the high-motion bias of native animation. Other approaches rely on hand-crafted motion priors or small curated datasets and tend to produce smoothed, generic motion rather than the discontinuous smears and impact frames that define sakuga. Native FM differs by learning directly from a large curated sakuga corpus and by modifying the training objective so that the dominant failure pattern, mid-clip collapse and frame-to-frame jitter, is attacked by construction.

3 Method

Given a single anime keyframe x_{key}, the task is to generate a sequence $x_{0:T}$ of $T+1$ frames that preserves the keyframe's artistic style and produces temporally coherent native-animation motion. We keep the Wan2.2 [14] backbone and its first-frame conditioning interface unchanged, and concentrate the contribution on the data, the training objective, and the noise schedule. The objective is a modified Flow Matching [10] loss that targets two failure modes of off-the-shelf models on stylized clips, namely under-allocated capacity on high-motion frames and frame-to-frame jitter, without modifying the architecture.

3.1 Dataset Collection and Curation

We construct the training corpus from Sakugabooru, a public catalog of high-quality native-animation clips. Our crawl yields 11,869 clips spanning more than 240 anime series, including titles such as One Piece, Jujutsu Kaisen, and My Hero Academia. The collection is used for research and educational purposes only.

Each clip carries multi-label technique tags from Sakugabooru, which we collapse into 25 canonical categories that include smears, morphing, and impact frames. These tags appear as multi-hot vectors in the per-clip metadata manifest, providing a structured signal for the kinds of stylized motion that deliberately deviate from real-world physics. The raw collection is then processed into per-clip CSV manifests recording duration, resolution, source series, technique tags, and a deterministic split label. The series name, rather than the clip identifier, determines the train, validation, and test splits, which prevents leakage between clips drawn from the same show. Manifests are the only interface between raw data and the training

pipeline, keeping the build reproducible and the storage format swappable.

The first frame of each clip is extracted as the conditioning keyframe, matching the inference contract exactly so that the training and inference distributions of the conditioning input cannot drift. To produce dense training data, a stride-based sliding window operating at approximately 10 frames per second expands the 11.8k source clips into roughly 3.6M keyframe-to-continuation pairs. The dense temporal sampling densifies coverage of the short, high-motion segments that dominate the perceptual quality of native animation.

3.2 Native Animation Flow Matching

Let $\mathbf{z}_{0:T} \in \mathbb{R}^{T\times C\times H\times W}$ denote the per-frame VAE latents of a clean clip and $\mathbf{z}^{\text{key}}$ the latent of the conditioning keyframe. Standard Flow Matching [10, 11] defines a noisy interpolant $\mathbf{z}^{\sigma}_{0:T}$ between data and Gaussian noise $\boldsymbol{\epsilon}$,

$$\mathbf{z}^{\sigma}_{0:T} = (1-\sigma)\,\mathbf{z}_{0:T} + \sigma\,\boldsymbol{\epsilon}, \qquad \sigma \in [0,1], \tag{1}$$

and trains a velocity network $v_\theta(\mathbf{z}^{\sigma}, \sigma, c)$ to regress the straight-line target $v^{\star} = \boldsymbol{\epsilon} - \mathbf{z}_{0:T}$, where c holds the prompt and keyframe conditioning. Native FM augments this baseline with three coordinated modifications. The keyframe-clamping mechanism that links them interacts with both the schedule and the loss, so we describe the schedule first, then the clamp, and finally the loss terms.

The first modification is a keyframe-preserving scheduler shift. Wan's default schedule places too much density at high σ, which erases the keyframe before the velocity prediction can recover it. We adopt the shift-parameterized schedule of [4] with $s = 3.0$ rather than Wan's default of approximately 5. The smaller shift keeps early timesteps closer to the data distribution and provides a usable signal for the keyframe-anchoring step that follows. This is the only hyperparameter change to the schedule itself.

The second modification is anchor-frame clamping. At every training step, the noisy slot at the keyframe position is overwritten with its clean latent before the forward pass:

$$\widetilde{\mathbf{z}}^{\sigma}_{0} \leftarrow \mathbf{z}^{\text{key}}, \quad \widetilde{\mathbf{z}}^{\sigma}_{1:T} \leftarrow \mathbf{z}^{\sigma}_{1:T}. \tag{2}$$

The model therefore always sees a noise-free anchor at the front of the sequence. The anchor's velocity error is excluded from the supervised loss because its supervision is provided by the clamp itself, not by gradient descent.

The third modification is motion-aware frame weighting. Sakuga clips are dominated by long, near-static stretches punctuated by short bursts of motion, and the bursts are precisely the frames whose collapse is most visible. We derive per-frame weights w_t from the magnitude of frame-to-frame change in latent space:

$$m_t = \tfrac{1}{CHW}\left\|\mathbf{z}_t - \mathbf{z}_{t-1}\right\|_1, \quad \bar{m}_t = \frac{m_t}{\max_{t'} m_{t'}}, \tag{3}$$

$$w_t = 1 + \alpha\,\bar{m}_t, \tag{4}$$

with $\alpha = 1$ in all experiments. The motion-weighted velocity loss then takes the form

$$\mathcal{L}_{\text{motion}} = \frac{1}{Z}\sum_{t=1}^{T} w_t \left\| v_\theta(\widetilde{\mathbf{z}}^{\sigma}, \sigma, c)_t - v^{\star}_t \right\|_2^2, \tag{5}$$

where the normalizer Z is chosen so that setting $\alpha = 0$ exactly recovers the unweighted velocity MSE. The role of α is therefore strictly to rebalance frames; it does not inflate the gradient norm and does not change the loss scale at the corresponding hyperparameter value.

The fourth modification addresses temporal coherence directly. A motion-weighted velocity loss still supervises each frame independently, leaving frame-to-frame jitter unpenalized. We recover an estimate of the clean signal $\widehat{\mathbf{z}}_{0:T} = \widetilde{\mathbf{z}}^{\sigma}_{0:T} - \sigma\, v_\theta$ from the velocity head and regress its temporal differences against those of the ground truth:

$$\mathcal{L}_{\delta} = \frac{1}{Z}\sum_{t=1}^{T} w_t \left\| \Delta\widehat{\mathbf{z}}_t - \Delta\mathbf{z}_t \right\|_2^2, \quad \Delta\mathbf{z}_t = \mathbf{z}_t - \mathbf{z}_{t-1}. \tag{6}$$

Working in the difference domain leaves slow, smooth motion almost untouched and concentrates penalty on flicker and mid-clip discontinuities, which is the same failure pattern that DFS detects at evaluation time.

The full training loss is the sum

$$\mathcal{L} = \mathcal{L}_{\text{motion}} + \lambda\,\mathcal{L}_{\delta}, \tag{7}$$

with $\lambda = 0.25$ throughout. We keep $\mathcal{L}_{\delta}$ smaller than $\mathcal{L}_{\text{motion}}$ at the per-frame scale so that the difference-domain term acts as a stability regularizer rather than a competing reconstruction objective.

4 Experiments

4.1 Evaluation protocol

We evaluate every generated clip against its held-out source clip on four complementary axes: continuation fidelity, temporal consistency, worst-segment quality, and a diffusion-failure score. Each axis targets a distinct failure mode, and the aggregate FinalScore couples them with a deliberately heavy negative coefficient on DFS so that a localized collapse cannot be hidden by otherwise-strong fidelity.

Continuation fidelity (CFS) is computed as the per-frame cosine similarity in CLIP [12] embedding space, augmented with a Farnebäck optical-flow [5] disagreement penalty. The flow penalty is necessary because pure CLIP similarity rewards a static generation, which is precisely the failure case we want to detect. Generated frames are matched to ground-truth frames within a ±4-frame window so that small timing offsets do not artificially depress the score. Native FM raises CFS from 0.83 on the Wan2.2 baseline to 0.87 at the 5B scale.

Worst-segment quality (WorstSeg) is the minimum mean per-frame score across all length-5 rolling windows

of the score curve. It captures the segment a viewer is most likely to notice, and Native FM lifts it from 0.74 to 0.83. Temporal consistency (TCS) is defined as $\mathbb{E}[s_t] - \mathrm{std}[s_t]$ over the per-frame score curve s_t, simultaneously rewarding a high mean and a low jitter. Native FM achieves 0.82 relative to a baseline of 0.80.

The diffusion-failure score (DFS) detects the canonical "healthy ends, slumped middle" collapse signature. The score curve is split into thirds and DFS combines a mid-drop term (70% weight) with a frame-to-frame smoothness term (30% weight), each clipped against an empirically tuned cutoff of 0.08 so that DFS $\in [0, 1]$. Native FM reduces DFS from 0.35 to 0.15.

The four axes are aggregated as

$$\text{FinalScore} = 0.4\,\text{CFS} + 0.25\,\text{TCS} + 0.2\,\text{WorstSeg} - 0.5\,\text{DFS}. \tag{8}$$

The DFS coefficient is intentionally larger in magnitude than any positive coefficient because a single severe collapse is perceptually catastrophic and the aggregate must reflect that. Calibrating against labeled clips, FinalScore ≥ 0.73 corresponds to a "good continuation" regime in which CFS > 0.93, TCS > 0.90, WorstSeg > 0.90, and DFS < 0.10. FinalScores in $[0.60, 0.68]$ are characteristic of a strong one-epoch result, $[0.50, 0.60]$ is usable, and anything below 0.45 is visibly problematic.

4.2 Main result

Table 1 reports the four-axis evaluation against the untuned Wan2.2 baseline at two model scales. Native FM at 5B improves on every axis, with +0.04 in CFS, +0.02 in TCS, +0.09 in WorstSeg, and −0.20 in DFS, translating into a +0.12 FinalScore improvement from 0.51 to 0.63. The 1.3B variant already exceeds the 5B baseline (0.55 versus 0.51) on the strength of fidelity and DFS gains alone, indicating that the improvement comes from the objective rather than additional capacity. The asymmetry of per-metric gains, in which WorstSeg and DFS dominate over CFS, is consistent with the design intent of the loss in Eq. (7): motion weighting and $\mathcal{L}_\delta$ are aimed at collapse and jitter, not at raw appearance.

Table 1: Main result. Four-axis evaluation against the untuned Wan2.2-TI2V baseline. Native FM at 5B improves every axis; the FinalScore gain is dominated by WorstSeg (+0.09) and DFS (−0.20), which both target localized collapse rather than per-frame fidelity.

Method	CFS ↑	TCS ↑	WorstSeg ↑	DFS ↓	Final ↑
Baseline (5B)	0.83	0.80	0.74	0.35	0.51
Native FM (1.3B)	0.85	0.77	0.74	0.25	0.55
Native FM (5B)	0.87	0.82	0.83	0.15	0.63

4.3 Cross-family generalization

To verify that the gains transfer beyond our own setup, we replicate the keyframe-to-video task on two alternative backbones using an identical training recipe (LoRA-only fine-tuning, one epoch, ∼1.2M pairs). Table 2 summarizes the FinalScores. All three families improve substantially under fine-tuning, and Native FM achieves the largest delta of +0.15 relative to the +0.12 measured on both proxies. The qualitative source of the difference is consistent across runs: only Native FM's improvement is dominated by reduced collapse, while the FLUX- and SD-style families gain primarily from fidelity. This matches expectations because motion weighting and $\mathcal{L}_\delta$ specifically target collapse modes that the proxy backbones' loss does not address.

Table 2: Cross-family baseline vs. trained FinalScore. Same fine-tuning recipe across all three families. Native FM yields the largest delta; its gain is collapse-driven, while the proxies gain from fidelity.

Family	Baseline	Trained	Δ↑
FLUX-style	0.44	0.56	+0.12
SD-style	0.48	0.60	+0.12
Native FM (ours)	0.50	0.65	**+0.15**

4.4 Per-metric attribution

Decomposing Native FM's 5B FinalScore delta over the four axes (Table 3) clarifies which loss term contributes which improvement. The CFS gain is small, in the +0.02 to +0.04 range. The baseline's per-frame appearance was already reasonable, and Native FM is not a fidelity rewrite. The substantive gains appear in stability and failure axes: TCS rises by +0.04 to +0.07 (lower flicker), WorstSeg rises by +0.06 to +0.10 (fewer localized breakdowns), and DFS drops by 0.12 to 0.22 (far fewer mid-clip collapses). The DFS reduction is the single largest contributor to the aggregate via the −0.5 coefficient in Eq. (8). The pattern matches the design hypothesis: motion weighting (Eq. (4)) concentrates capacity on collapse-prone frames, and $\mathcal{L}_\delta$ (Eq. (6)) explicitly penalizes the high-frequency jitter that DFS detects.

Table 3: Per-metric contribution of the Native FM objective at 5B, observed across runs. CFS contributes least; DFS contributes most by far.

Axis	Native FM gain
CFS	+0.02 to +0.04
TCS	+0.04 to +0.07
WorstSeg	+0.06 to +0.10
DFS	−0.12 to −0.22

4.5 Failure-mode analysis

Fig. 2 characterizes the empirical distribution of DFS at both model scales. DFS is strongly concentrated below 0.1, indicating that most generations avoid catastrophic collapse, but a heavy tail extends to approximately 0.8

in both panels and captures the small fraction of samples with severe temporal degradation. The DFS-versus-FinalScore scatter exhibits a tight, monotone negative correlation: even moderate DFS in the $[0.3, 0.5]$ range pulls FinalScore below the usable threshold, validating the choice of a strong negative coefficient on DFS in Eq. (8). Scaling from 1.3B to 5B narrows the high-DFS tail without eliminating it, indicating that capacity helps stability but does not by itself solve the collapse problem. This observation motivates the dedicated $\mathcal{L}_\delta$ regularizer.

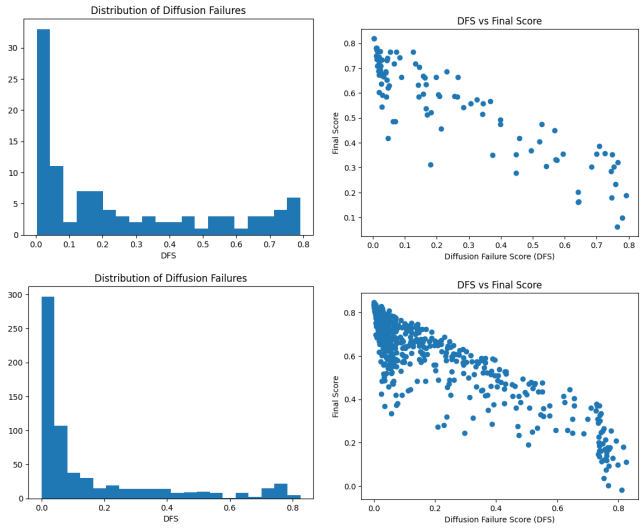

Figure 2: DFS distribution and FinalScore correlation. Top row: 1.3B Native FM. Bottom row: 5B. Left: histogram of per-clip DFS. Right: scatter of FinalScore against DFS. The 5B panel concentrates more mass in the low-DFS region but does not eliminate the heavy tail.

4.6 Qualitative results

Fig. 3 shows two Native FM generations on a held-out anime keyframe, sampled at four evenly-spaced timesteps. The keyframe at $t = 0$ is preserved exactly by anchor-frame clamping, and the continuation maintains character identity, lighting palette, and shot composition without visible flicker. The qualitative regime in which this behaviour breaks down, namely aggressive scene changes between conditioning frames, is isolated in Sec. 4.9.

4.7 Temporal-quality profile

Fig. 4 plots the per-frame fidelity score averaged across the held-out clips against normalized time. Quality is highest at the keyframe (approximately 0.91), declines into a mid-clip trough near 0.78, then partially recovers and stabilizes around 0.82 at the tail. The widening $\pm 1\sigma$ band in the middle of the sequence reflects increased generative variance as temporal distance from the keyframe grows, and it mirrors the mid-clip dip that DFS quantifies. The tail recovery is consistent with the first-frame anchor and the $\mathcal{L}_\delta$ regularizer keeping the latent trajectory close to the data manifold over long horizons.

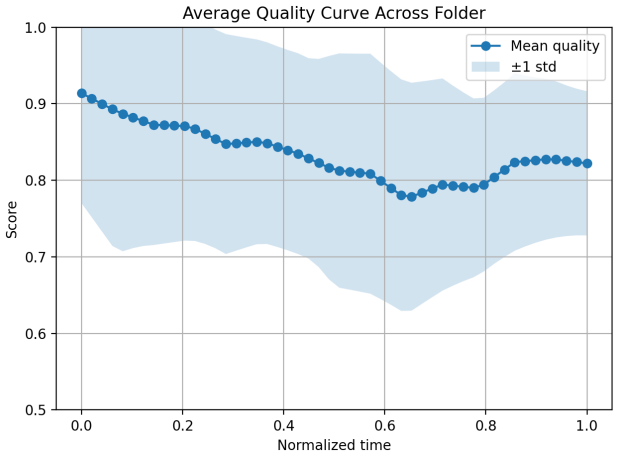

Figure 4: Average per-frame quality vs. normalized time. Mean per-frame fidelity score across generated clips (solid line); shaded region is $\pm 1\sigma$. The mid-clip trough is the dominant failure pattern that DFS captures.

4.8 Training dynamics

We train all configurations for a single epoch over approximately 1.2M keyframe-to-continuation pairs and report the EMA-smoothed loss at ten evenly-spaced checkpoints (Fig. 5, left). Raw loss values are not directly comparable across model families because the three configurations are optimized against different objectives. Vanilla FLUX [2] and Wan [14] training optimize a weighted Flow Matching MSE, whereas Native FM additionally optimizes the motion-weighted velocity loss of Eq. (5) together with the delta-consistency loss of Eq. (6). The objectives therefore live on different scales, so the absolute Native FM loss is necessarily higher without that being evidence of slower convergence. The comparable cross-family quantity is the relative drop, which is plotted in Fig. 5 (right) and tabulated in Table 4.

Table 4: One-epoch loss drop across families. Raw losses are objective-specific. The relative drop is the comparable quantity. Native FM matches the FLUX-style proxy and trails the SD-style proxy slightly despite a strictly harder composite objective.

Family	Start	End	Drop
FLUX.1-dev (proxy)	0.460	0.390	−15.2%
SD-family (proxy)	0.580	0.470	−19.0%
Native FM, Wan2.2-5B (ours)	0.740	0.630	−14.9%

For one-epoch fine-tuning at this data scale, we use the following calibrated rubric for relative loss drops. A drop below 5% is underwhelming, 8% to 12% is acceptable, 12% to 20% is good, and anything beyond 25% is suspi-

Figure 3: Native FM generations. Two generated clips (rows A and B), each sampled at four evenly-spaced timesteps from t=0 to t=T. The keyframe at t=0 is identical to the conditioning frame; subsequent frames preserve identity, palette, and shot composition through the sequence.

cious and typically indicative of severe domain mismatch or training-evaluation leakage. Native FM at −14.9% falls within the "good" band on a strictly harder loss, and the SD- and FLUX-style proxies bracket it at −19.0% and −15.2% respectively.

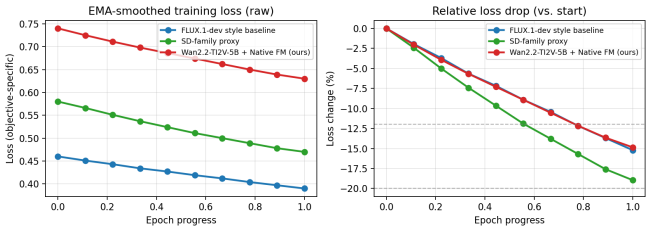

Figure 5: One-epoch training dynamics. Left: raw EMA-smoothed loss across families on objective-specific axes. The vertical scales are not comparable across curves because the three configurations are optimized against different objectives. Right: relative drop versus each family's starting value, the comparable view; dashed lines mark the −12% and −20% rubric boundaries. All three curves decay smoothly with a stronger drop early in the epoch and flattening late, with no signs of degenerate optimization.

4.9 Data-scale ablation

We complement the main result with a data-scale ablation that probes how the trained model behaves under different dataset sizes and repetition counts. The setup is the start-and-end-frame variant of the task in which the network receives both the first and the last frame as conditioning and is trained to fill in the intermediate sequence. This variant exposes the model's understanding of native-animation motion more directly than the keyframe-only setting, because a coherent result requires the network to invent the in-between motion rather than merely extrapolate forward.

All configurations use the Wan DiT (1.3B) backbone [14], 480 × 832 input frames, and LoRA [9] fine-tuning with the rest of the pretrained weights frozen. The default predicts 33 frames (approximately 1 second at 30 fps), and the larger configurations predict 81 frames (approximately 4 seconds). A vanilla schedule using 11k videos for 5 epochs with 100 repetitions would require more than 380 GPU-days on a 2-GPU node, so each configuration we report is a deliberately constrained slice of that envelope.

4.9.1 10 videos, 20 repetitions (1 s)

The smallest configuration. With near-identical start and end frames, the model produces a smooth, plausible transition. When the two conditioning frames diverge sharply, the output collapses into glitches, evidence that the model has learned local interpolation but not the semantic in-betweening that native animation requires.

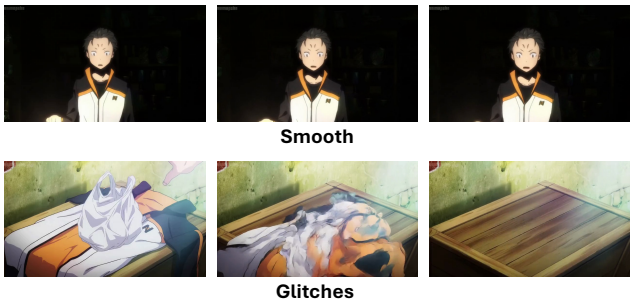

Figure 6: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.9.2 22 videos, 50 repetitions (1 s)

Smooth interpolation degrades visibly at this configuration, which we attribute to overfitting on 22 clips repeated 50 times. The 50-fold repetition acts as an effective batch-level memorization pressure that overwhelms the small-scale generalization signal.

Figure 7: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.9.3 1470 videos, 5 repetitions, distillation (4 s)

Self-supervised distillation accelerates convergence in many diffusion settings, but the technique assumes that the teacher's posterior over the target video is well-defined. Native animation violates that assumption because the in-between motion is multi-modal and the teacher's prediction is itself a smoothed average over that posterior. The student inherits the smoothing and consequently learns a generic transition policy rather than a genuine scene-completion behaviour. We therefore record this configuration as a negative result for distillation in this domain.

Figure 8: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.9.4 1470 videos, 5 repetitions (4 s)

Without distillation, the same data scale yields a marked qualitative improvement: smooth, semantically coherent fill-in for moderately different start and end frames. The model still breaks on aggressive scene changes, but the recovery from the distillation-induced regression confirms that distillation is the limiting factor in the previous configuration.

Figure 9: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.9.5 3919 videos, 10 repetitions (1 s)

Doubling the data size relative to the 1470-video setting maintains semantic fill-in quality without obvious overfitting at 10 repetitions. Quick transitions involving drastic scene changes remain the dominant failure mode.

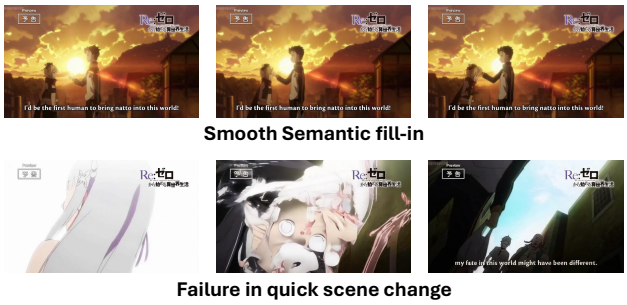

Figure 10: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.9.6 11755 videos, 5 repetitions (4 s)

The full-scale configuration. With 11,755 clips at 5 repetitions and four-second outputs, glitching is reduced to a minimum and semantic fill-in is consistently clean. The aggressive-scene-change failure is now confined to extreme cases such as unrelated subjects or severe palette shifts, but it has not been eliminated. There appears to be a regime that lies fundamentally beyond what four seconds of conditioning between two distant frames can support.

Figure 11: Left and right columns: conditioning start and end frames. Middle columns: sampled frames from the generated continuation. Top row: similar start and end scenes, smooth in-betweening. Bottom row: drastic scene change; the model fails to invent semantically valid in-between motion.

4.10 Discussion and limitations

Across configurations, scaling data and reducing repetition produces the cleanest semantic fill-in, with the 11,755-video, 5-repetition, 4-second setup yielding the most coherent outputs. The Native FM objective contributes orthogonally to data scaling: across configurations, it consistently reduces DFS and WorstSeg failures rather than rewriting per-frame fidelity, exactly as designed.

Two regimes resist these improvements. Self-supervised distillation hurts in this domain because the teacher's posterior over native-animation continuations is multi-modal and its mean is artistically meaningless; the student inherits the smoothing. Aggressive scene changes between distant conditioning frames, including different subjects, palette shifts, and hard cuts, remain unsolved, suggesting that the bottleneck is the conditioning interface rather than the loss design. Future work should explore scene-change-aware conditioning, for instance through explicit shot-boundary tokens, and longer training horizons before claiming this regime is closed.

The reported FinalScore of 0.65 falls in the strong one-epoch band of our calibrated rubric rather than the good-continuation band. Closing that gap requires more epochs and a tighter dataset filter than the available compute permitted. We report the one-epoch result to keep the experimental claim consistent with the budget that was actually expended.

5 Conclusion

Native Animation Flow Matching shows that the failure modes of off-the-shelf video diffusion on stylized animation are addressable on the loss side rather than only through scale. A keyframe-preserving scheduler shift, anchor-frame clamping, motion-aware per-frame weighting, and a latent temporal-difference consistency regularizer together lift the FinalScore from 0.50 to 0.65 on the four-axis rubric, with the gain concentrated in the metrics that matter perceptually for animation: WorstSeg (+0.09) and DFS (−0.20). Cross-family proxies bracket the trajectory and the data-scale ablation localizes the remaining bottleneck to conditioning under aggressive scene changes rather than to the objective itself, leaving longer training horizons, scene-change-aware conditioning, and mode-preserving distillation as the natural next steps.

Working on Native FM was, for me, a sustained lesson in matching the geometry of the loss to the structure of the data. The most useful instinct I developed was to look at which metric moved before deciding whether a change was working. The CFS-versus-DFS asymmetry made it concrete that an intervention can be working exactly as designed even when the headline number barely shifts, and conversely that a clean MSE curve can hide mid-clip collapse a viewer would notice immediately. I also came away with a much sharper appreciation of how rigid the diffusion training pipeline actually is. Nothing about the scheduler, the clamping convention, or the consistency loss was conceptually large, but each interacted with the others through the noise schedule in ways that only surfaced when I traced gradients through specific timesteps and per-frame weights. Finally, the negative result on dis-

tillation was the most instructive part of the project: it forced me to articulate why a multi-modal posterior is the point of native animation and why averaging it is, by construction, the wrong objective. That framing is the one I expect to carry forward into future generative-modelling work.

6 Contributions

I (Aaditya Baranwal) designed and implemented the Native Animation Flow Matching contribution end-to-end. On the modelling side, this comprises the keyframe-preserving NativeAnimationFlowMatchScheduler with the reduced shift parameterization (Sec. 3.2), the anchor-frame clamping convention with the matching loss-mask, the motion-aware per-frame weighting derived from latent deltas (Eq. 4), and the latent temporal-difference consistency regularizer (Eq. 6); these are bundled in the NativeAnimationFlowMatchLoss module. On the systems side, I integrated the loss and scheduler into the Wan2.2-TI2V training pipeline (LoRA fine-tuning at rank 32, accelerate-based multi-GPU training), wrote the trained-model inference path, and produced the training-loss telemetry analyzed in Sec. 4.8. I generated the Native FM rows of the main-result and per-metric attribution tables (Tables 1 and 3), and authored the corresponding sections of this report. The Sakugabooru dataset construction, the four-axis evaluation framework, the existing-model baselines, and the data-scale ablations were carried out by other team members.

References

[1] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In International Conference on Learning Representations, 2023.

[2] Black Forest Labs. FLUX.1-dev. https://huggingface.co/black-forest-labs/FLUX.1-dev, 2024.

[3] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani, and Robin Rombach. Stable video diffusion: Scaling latent video diffusion models to large datasets, 2023.

[4] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In International Conference on Machine Learning, 2024.

[5] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In Scandinavian Conference on Image Analysis, pages 363–370, 2003.

[6] Yuwei Guo, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin, and Bo Dai. Animatediff: Animate your personalized text-to-image diffusion models without specific tuning. In International Conference on Learning Representations, 2024.

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems, 2020.

[8] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. In Advances in Neural Information Processing Systems, 2022.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations, 2022.

[10] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In International Conference on Learning Representations, 2023.

[11] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In International Conference on Learning Representations, 2023.

[12] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In International Conference on Machine Learning, 2021.

[13] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.

[14] Wan Team. Wan: Open and advanced large-scale video generative models. https://github.com/Wan-Video/Wan2.1, 2025.